
Fast End-to-end Retrieval using Retrieval Layers

Rohun Agrawal
Columbia University
rohun.agrawal@columbia.edu

Micah Goldblum
Columbia University
micah.g@columbia.edu

Abstract

Retrieval-augmented generation typically treats retrieval as a scaffold outside the model: a frozen or separately-trained retriever selects passages, which are then prefilled into the context window at quadratic attention cost and without being optimized against the generation objective. We instead make retrieval a native, end-to-end learned component. A *retrieval layer* encodes external documents into compact key and value vectors and reads from them via sparse top- k attention inside the transformer, with the search mechanism and document representations trained jointly with the language model against the generation loss. We instantiate a retrieval layer inside a single middle layer of a Qwen3-4B decoder, with document features from a jointly fine-tuned embedding model, and train the system on a hard-negative multi-hop/open-domain QA mixture with an auxiliary InfoNCE document-access loss. On a long-document benchmark (QASPER, a 6.3M-token corpus), the retrieval layer matches the downstream QA accuracy of RAG@5 at $2.0\times$ lower end-to-end serving latency.

1 Introduction

Retrieval-augmented generation (RAG) is the dominant mechanism for grounding large language models in knowledge that changes after pretraining or that is too large to memorize in weights [Lewis et al., 2020]. In practice, RAG is implemented as a scaffold around the model rather than as a learned part of it: a frozen or separately-trained retriever selects passages, and those passages are prefilled into the model’s context window. This design has two costs. First, it is computationally wasteful: attention over prefilled documents scales quadratically with the number of retrieved tokens, most of which are irrelevant to the query. Second, it is brittle. Because retrieval is not optimized against the generation objective, it degrades on multi-hop queries that require synthesizing information spread across several documents [Tang and Yang, 2024, Trivedi et al., 2022].

We study an alternative in which retrieval is a native, end-to-end learned component of the architecture rather than a post-hoc scaffold. A *retrieval layer* encodes external documents into compact key and value vectors and reads from them via sparse top- k attention, with the search mechanism and the underlying representations trained jointly with the language model against the generation loss. Because only a small number of vectors are attended to per query, retrieval layers avoid the quadratic prefill cost of standard RAG while remaining differentiable end to end.

We instantiate a retrieval layer inside a single middle layer of a Qwen3-4B decoder. The backbone is frozen through a memory-warmup schedule and then fully fine-tuned; document key/value representations come from a jointly fine-tuned pretrained embedding model. We train the whole system against the generation loss on a hard-negative multi-hop/open-domain QA mixture with chain-of-thought supervision. On a long-document benchmark (QASPER [Dasigi et al., 2021], a 6.3M-token corpus), the retrieval layer reaches QA accuracy comparable to RAG@5 at $2.0\times$ lower end-to-end serving latency, and much lower latency than a KV-cache-compression baseline (SnapKV [Li et al., 2024]).

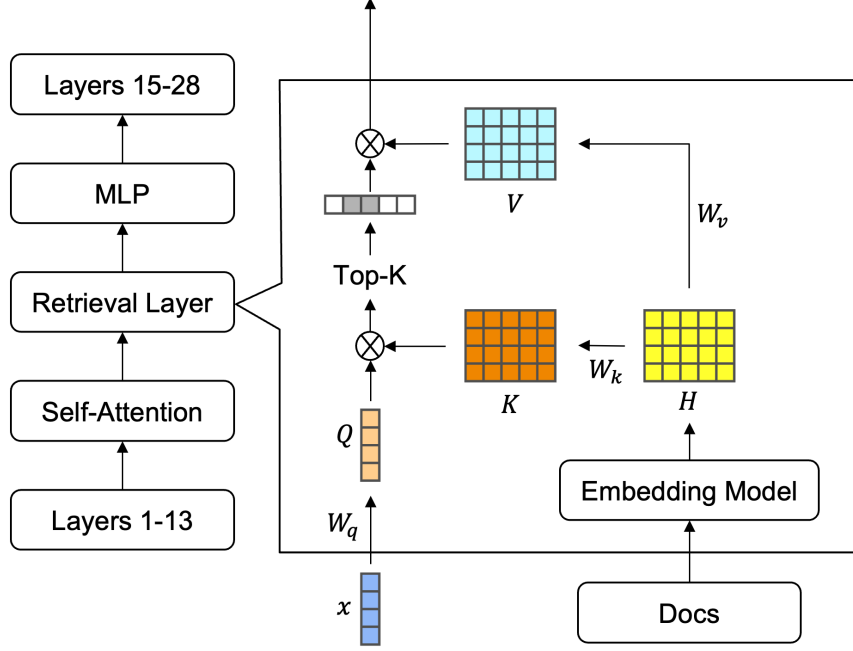


Figure 1: The retrieval layer inserted into a Qwen3-4B decoder. A hidden state x is projected to a query q , which attends sparsely (top- k) over key/value vectors K, V derived from document features H . H is produced by an embedding model run over the document collection, and the projections W_q, W_k, W_v are trained jointly with the rest of the network.

2 Related Work

Prior approaches to giving language models access to external knowledge fall into three families. Retrieve-then-prefill methods prefill retrieved text and pay quadratic attention cost over it. RAG itself [Lewis et al., 2020], RETRO [Borgeaud et al., 2022], and REALM [Guu et al., 2020] all use retrievers that are frozen or trained with an auxiliary objective rather than jointly with the generation loss. Cached-representation methods read cached vectors instead: k NN-LM [Khandelwal et al., 2020] interpolates output probabilities with a nearest-neighbor lookup, while Memorizing Transformers [Wu et al., 2022] and LongMem [Wang et al., 2023] attend over cached key-value pairs without learning the search mechanism or the representations end to end. Corpus-amortization methods write documents into model weights or fast weights instead, as in differentiable search indices [Tay et al., 2022], Cartridges [Eyuboglu et al., 2025], and test-time training [Tandon et al., 2025, Behrouz et al., 2026]. These entangle document content with model parameters, so clean removal of a document is difficult.

Retrieval layers build most directly on *memory layers* [Berges et al., 2025, Huang et al., 2025], which store knowledge in sparsely-addressed key-value memory rather than in dense feedforward weights. We extend this mechanism from a fixed bank of trained-from-scratch memory vectors to representations extracted from a corpus of external documents, so the same sparse-attention machinery serves as a retrieval mechanism rather than a static store. This is conceptually related to conditional-computation mechanisms such as Mixture-of-Depths [Raposo et al., 2024], which route computation rather than knowledge.

3 Retrieval Layers

We adopt the following formulation which we visualize in Figure 2. Let $x \in \mathbb{R}^d$ be a hidden state at the retrieval layer, and let a document collection be encoded by a single feature extractor h_ϕ , applied per token, into document features $H = \text{concat}\{h_\phi(\text{doc})_i\} \in \mathbb{R}^{N \times d'}$. Learned linear

projections $W_k, W_v \in \mathbb{R}^{d' \times d}$ turn these features into keys and values, $K = HW_k \in \mathbb{R}^{N \times d}$ and $V = HW_v \in \mathbb{R}^{N \times d}$. A query $q = W_q x$ selects a sparse subset of tokens across the whole collection:

$$\mathcal{I} = \text{SelectTopk}(Kq), \quad s = \text{Softmax}(K_{\mathcal{I}}q), \quad y = sV_{\mathcal{I}}. \quad (1)$$

h_ϕ and the projections W_k, W_v are trained jointly with the rest of the language model against the end-task loss, so the search itself is learned, not just what is done with the retrieved content. At inference, $h_\phi(\text{doc})$, and therefore K and V , do not depend on the current query and can be computed once and cached, so adding, removing, or editing a document only requires re-encoding that document rather than retraining or updating a context window.

Document-access loss. A document-access auxiliary loss is applied to every retrieval query, following a multi-positive InfoNCE objective over the retrieval logits. For a query q at a given head, let $z_m = k_m^\top q$ be its logit against bank slot $m \in \{1, \dots, M\}$ (the same logits used for top- k selection); let $\mathcal{P}(q) \subseteq \{1, \dots, M\}$ be the set of token positions belonging to that query’s own gold document(s); and let τ be a temperature. The loss is

$$\mathcal{L}_{\text{doc}}(q) = \log \sum_{m=1}^M \exp(z_m/\tau) - \log \sum_{m \in \mathcal{P}(q)} \exp(z_m/\tau) = -\log \frac{\sum_{m \in \mathcal{P}(q)} \exp(z_m/\tau)}{\sum_{m=1}^M \exp(z_m/\tau)}, \quad (2)$$

averaged over all query positions and retrieval heads. The negative pool in the denominator is every scored token slot in the training batch: the query’s own hard-negative distractor documents together with every other example’s documents in the batch, not merely the tokens retrieved at top- k . The objective therefore supervises the ranking of the full bank rather than only what ends up read, and is added to the cross-entropy generation loss during training.

4 Experimental Setup

Model. The backbone is Qwen3-4B. A single retrieval layer is inserted after the self-attention sub-block of layer 14, rather than replacing the FFN. Document features h_ϕ come from a pretrained Qwen3-Embedding-0.6B encoder, run bidirectionally (not causally); learned linear projections turn these features into the key and value vectors used by the retrieval layer. Retrieval uses 4 heads over a bank of 65,536 key/value slots (key and value dimension 1024 each). Because the number of scored key/value pairs is large, we select $k = 64$ using an approximate top- k (`jax.lax.approx_max_k`, `recall@64` ≈ 0.99) rather than exact top- k , applied identically at train and eval time. Training follows a 4-stage schedule: for the first three stages the backbone is frozen and only the retrieval layer, its query/key/value projections, and the document encoder are trained; the final stage additionally unfreezes the backbone and fine-tunes it jointly with the rest of the model under a cosine-decayed learning rate. The document-access loss (Eq. 2) is added with a constant weight of $\lambda_{\text{doc}} = 0.1$ throughout all four stages.

Data. Training data is a mixture of four HF-hosted hard-negative QA sources: ScienceQA, an open-domain “DiverseQA” set, a combined TriviaQA/HotpotQA/NQ/SQuAD/MS MARCO mixture, and a multi-hop QA set. Each example provides a query, a chain-of-thought answer, and hard-negative distractor documents (cosine similarity ≥ 0.95 to the gold document under the source retriever, at least 2 distractors per example). Documents are chunked into 16 chunks of 256 tokens each; total input sequence length is 512 and batch size is 16. We finetune for 100k steps.

Baselines. We compare against (i) conventional RAG: Qwen3-Embedding-0.6B embeds each candidate document, and the top-5 by cosine similarity are prefilled verbatim into the Qwen3-4B prompt (RAG@5); and (ii) SnapKV [Li et al., 2024], a KV-cache-compression baseline that stuffs a compressed representation of the full corpus into context rather than retrieving a sparse subset of it.

5 Results

Overall, we find that our Retrieval Layer Model (RLM) achieves comparable multi-hop QA accuracy with lower serving latency on long documents. Figure 2 compares serving latency on QASPER (1,169 papers, 6.3M tokens, whole-corpus retrieval) against downstream QA accuracy on MuSiQue (2,048-document corpus, $n = 128$, Qwen3-4B judge). These are two different benchmarks: no checkpoint here was trained on long documents, so accuracy on QASPER itself would be out-of-distribution.

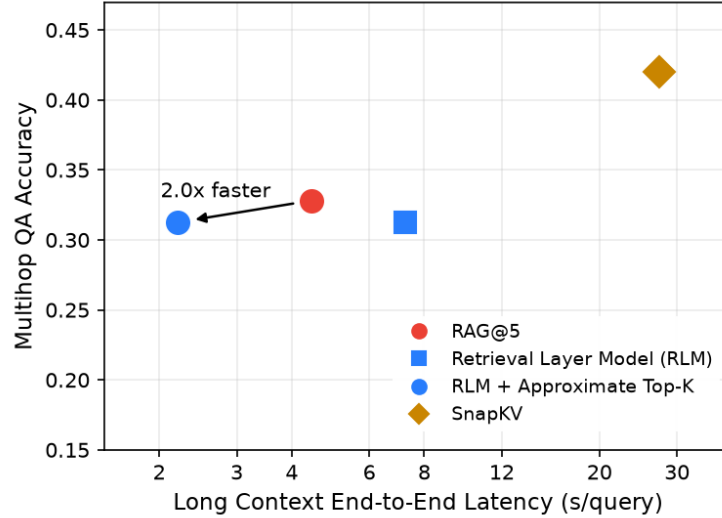


Figure 2: Long-document serving latency (QASPER, x-axis, log scale) vs. downstream QA accuracy (MuSiQue, y-axis) for RAG@5, the retrieval layer (RLM) with exact top- k , RLM with approximate top- k , and SnapKV.

RAG@5 must prefill a 27,084-token prompt per query; the retrieval layer prefills only the 64-token question, regardless of corpus size. At comparable MuSiQue accuracy (0.328 for RAG@5 versus 0.313 for the retrieval layer with approximate top- k), the retrieval layer is $2.0\times$ faster end-to-end (2.198 versus 4.430 s/query).

The approximate top- k optimization improves throughput substantially: exact top- k alone is slower than RAG@5 (7.259 s/query) at this bank size (6.3M slots). A KV-cache-compression baseline (SnapKV, run at $320\times$ KV-cache compression on this corpus) reaches higher accuracy (0.42), but only at $\sim 12\times$ the latency of the retrieval layer (27.3 versus 2.198 s/query).

6 Conclusion

We implement a retrieval layer inside a Qwen3-4B decoder: a single sparse top- k attention layer over jointly-trained document key/value representations, supervised end to end with an auxiliary InfoNCE document-access loss. We train it on a hard-negative multi-hop/open-domain QA mixture. On a long-document serving benchmark, it reaches downstream QA accuracy comparable to conventional RAG at $2.0\times$ lower end-to-end latency, and at a fraction of the latency of a KV-cache-compression baseline. An interesting direction of future work is to combine RAG and our RLM by doing a generous initial round of RAG and only populating the retrieval layer with the retrieved documents.

References

- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *Advances in Neural Information Processing Systems*, 38:113506–113543, 2026.
- Vincent-Pierre Berges, Barlas Oguz, Daniel Haziza, Wen-tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. In *Forty-second International Conference on Machine Learning*, 2025.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, pages 2206–2240, 2022.

- Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610, 2021.
- Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Ré. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*, 2025.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, pages 3929–3938, 2020.
- Zihao Huang, Qiyang Min, Hongzhi Huang, Yutao Zeng, Defa Zhu, Ran Guo, and Zhou Xun. Ultra-sparse memory network. In *International Conference on Learning Representations*, pages 86557–86575, 2025.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. 2024.
- Arnub Tandon, Karan Dalal, Xinhao Li, Daniel Kocaja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context. *arXiv preprint arXiv:2512.23675*, 2025.
- Yixuan Tang and Yi Yang. Multihop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. In *First Conference on Language Modeling*, 2024.
- Yi Tay, Vinh Q Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems*, 35:21831–21843, 2022.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems*, 36:74530–74543, 2023.
- Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.